



COURSE NOTES

Anshu React

Theory Notes & Interview Q&A



by Anshu Saini

EPISODES 01 - 08 · 13

Contents

Anshu React — Theory Notes

01	Inception	8 Q&A	3
02	Igniting Our App	14 Q&A	6
03	Laying the Foundation	4 Q&A	11
04	Talk is Cheap, show me the code	12 Q&A	14
05	Let's get Hooked!	3 Q&A	20
06	Exploring the world	11 Q&A	24
07	Finding the Path	5 Q&A	29
08	Let's get Classy	5 Q&A	32
13	Time for the Test		34

01

EPISODE 01

Inception

Q What is **Emmet** ?

A **Emmet** is the essential toolkit for web-developers. It allows you to **type shortcuts** that are then expanded into full pieces of code for writing **HTML and CSS**, based on an abbreviation structure most developers already use that expands into full-fledged HTML markup and CSS rules.

Q What is **CDN** ? Why do we use it?

A A **content delivery network (CDN)** refers to a geographically distributed group of servers that work together to provide fast delivery of Internet content. The goal is to provide high availability and performance by distributing the service spatially relative to end users.

Q Why is **React known as React** ?

A And it's called **React** because it **reacts**. It was developed by Facebook (a site that CONSTANTLY updates their data) to improve the user interface development and more effectively change (REACT to) what the user sees when they're doing things like mouse clicking, submitting and typing.

Q What is **crossorigin in script tag** ?

A The **crossorigin** attribute sets the mode of the request to an HTTP CORS Request. The purpose of crossorigin attribute is used to share the resources from one domain to another domain. Basically, it is used to handle the CORS request. It is used to handle the CORS request that checks whether it is safe to allow for sharing the resources from other domains.

Syntax

```
<script crossorigin="anonymous|use-credentials">
```

HTML

Q What is difference between **React** and **ReactDOM** ?

A **React** is a JavaScript library for building User Interfaces whereas **ReactDOM** is also JavaScript library that allows **React to interact with the DOM** .

The react package contains **React.createElement()** , **React.Component** , **React.Children** , and other helpers related to elements and component classes. You can think of these as the isomorphic or universal helpers that you need to build components. The react-dom package contains **ReactDOM.render()** , and in react-dom/server we have server-side rendering support with **ReactDOMServer.renderToString()** and **ReactDOMServer.renderToStaticMarkup()** .

Q What is difference between **react.development.js** and **react.production.js** files via **CDN** ?

A **Development** is the stage of an application before it's made public while **production** is the term used for the same application when it's made **public** . **Development build** is several times (maybe 3-5x) **slower** than the **production build** .

Q What are **async** and **defer** attributes in **<script>** tag?

A **Async** - The async attribute is a **boolean attribute** . The script is downloaded in **parallel(in the background)** to parsing the page, and **executed as soon** as it is available (do not block HTML DOM construction during downloading process) and don't wait for anything.

Syntax

```
<script async src="demo_async.js"></script>
```

HTML

Defer - The defer attribute is a **boolean attribute** . The script is downloaded in **parallel(in the background)** to parsing the page, and **executed after the page** has finished parsing (when browser finished DOM construction). The **defer attribute** tells the browser **not to wait for the script** . Instead, the browser will continue to process the HTML, build DOM.

Syntax

```
<script defer src="demo_defer.js"></script>
```

HTML

Difference between a **Library and Framework** ?

 A framework is a set of pre-written code that provides a structure for developing software applications. A library, on the other hand, is a collection of pre-written code that can be used to perform specific tasks.

A **library** is a collection of packages that perform specific operations whereas a **framework** contains the basic flow and architecture of an application. The major difference between them is the complexity. Libraries contain a number of methods that a developer can just call whenever they write code. React js is library and Angular is Framework.

The **framework** provides the flow of a software application and tells the developer what it needs and calls the code provided by the developer as required. If a **library** is used, the application calls the code from the library.

02

EPISODE 02

Igniting Our App

Q What is `npm`?

A `npm` is the world's largest Software Registry. The registry contains over 800,000 code packages. Open-source developers use `npm` to share software.

`npm` is lots of things:

- `npm` is the package manager for Node.js. It was created in 2009 as an open source project to help JavaScript developers easily share packaged modules of code.
- The `npm` Registry is a public collection of packages of open-source code for `Node.js`, `front-end web apps`, `mobile apps`, `robots`, `routers`, and countless other needs of the JavaScript community.
- `npm` alternative is `yarn`

How to initialize `npm`?

```
npm init
```

TERMINAL

`npm init -y` can be used to skip the setup step, `npm` takes care of it and creates the `package.json` json file automatically, but without configurations.

- `npm` is the command line client that allows developers to install and publish those packages.

Q What is `Parcel/Webpack`? Why do we need it?

A `Parcel` and `webpack` are the bundlers used mostly for `JavaScript` or `Typescript` code that helps you to `minify, clean, and make your code compact` so that it becomes easier to send a request or receive the response from the server when it usually takes you to transfer multiple files without using any bundler for loading the page of your application. Both of these bundlers substantially reduce the time it takes for the transfer of data and files to the server

from the application. Along with that both bundlers parcel and webpack remove the unnecessary comments, new lines, any kind of block delimiters, and white spaces while the functionality of the code remains unchanged.

Use of **Parcel/Webpack**: Module bundlers are the way to organize and combine many files of JavaScript code into one file. A JavaScript bundler can be used when your project becomes too large for a single file or when you're working with libraries that have multiple dependencies.

installation commands:

- Install:

```
npm install -D parcel
```

TERMINAL

-D is used for development and as a development dependency.

- Parcel Commands :
 - For development build:

```
npx parcel <entry_point>
```

TERMINAL

- For production build :

```
npx parcel build <entry_point>
```

TERMINAL

Q Why is **.parcel-cache** folder?

A **cache folder** (or **.parcel-cache in parcel v2**) stores information about your project when parcel builds it, so that when it rebuilds, it doesn't have to re-parse and re-analyze everything from scratch. It's a key reason why parcel can be so fast in development mode.

Q What is **npx**?

A **npx** stands for **Node Package eXecute**. It is simply an **npm** package runner. It allows developers to execute any Javascript Package available on the npm registry without even installing it. npx is installed automatically with npm version 5.2.

Q What is difference between **dependencies vs devDependencies** ?

A **"dependencies"** : Packages required by your application in production.

"devDependencies" : Packages that are only needed for local development and testing.

Q What is Tree Shaking? in Parcel?

A **Tree shaking** , also known as **dead code elimination** , is the practice of **removing unused code in your production build** . It's important to ship as little code to your end-users as possible. By statically analyzing our source code, we can determine what's not being used and exclude it from our final bundle.

Q What is Hot Module Replacement?

A **Hot Module Replacement (HMR)** exchanges, adds, or removes modules while an application is running, without a full reload. This can significantly speed up development in a few ways: Retain application state which is lost during a full reload. Save valuable development time by only updating what's changed.

Q List down your favourite 5 superpowers of Parcel and describe any 3 of them in your own words.

Parcel features:

- Dev Build - parcel provides us develop build
- Local Server - parcel also provides us a local server, which can be used to see live changes in our application.
- HMR = Hot Module Replacement - exchanges, adds, or removes modules while an application is running, without a full reload

and some of more cool features of Parcel are:

- File Watching Algorithm - written in C++
- Caching - Faster Builds
- Image Optimization
- Minification
- Bundling
- Compress

- Consistent Hashing
- Code Splitting
- Differential Bundling - support older browsers
- Diagnostic
- Error Handling
- HTTPs
- Tree Shaking - remove unused code
- Different Build for dev and prod bundles

Q What is `.gitignore`? What should we add and not add into it?

A gitignore file tells Git which files to ignore when committing your project to the GitHub repository. gitignore is located in the root directory of your repo. / will ignore directories with the name.

In our code we shouldn't add the files, which we can re-generate in future e.g, `node_modules`, `dist` etc.

Q What is the difference between `package.json` and `package-lock.json` files?

A Both of these files have the same format, and perform similar functions in the root of a project. The difference is that `package-lock.json` cannot be published, and it will be ignored if found in any place other than the root project.

The package.json is used for more than dependencies - like defining project properties, description, author & license information, scripts, etc. The package-lock.json is solely used to lock dependencies to a specific version number.

Q Why should I not modify `package-lock.json`?

A It is a generated file and is not designed to be manually edited. Its purpose is to track the entire tree of dependencies (including dependencies of dependencies) and the exact version of each dependency. You should commit `package-lock.json` to your code repository

You should avoid updating the `package.json` manually since it could break the synchronization between `package.json` and `package-lock.json`.

Q What is `node_modules`? Is it a good idea to push that on git?

A The `node_modules` folder contains generated code. This is not code you've written and you should never make any updates to the files inside Node modules because there's a pretty good chance they'll get overwritten next time you install some modules.

It is better to not commit the `node_modules` folder, and instead add it to your `.gitignore` file.

Here are all the reasons why you shouldn't commit it: The `node_modules` folder has a massive size (up to Gigabytes). It is easy to recreate the `node_modules` folder via `packages.json`

Q What is the `dist` folder?

A The `/dist` stands for distributable. The `/dist` folder contains the minimized version of the source code. The code present in the `/dist` folder is actually the code which is used on production web applications.

Parcel's default directory for your output is named `dist`. The `--dist-dir public` tag defines the output folder for your production files and is named `public` to avoid confusion with the `dist` default directory.

Q What is `browserslists`?

A Browserslist defines and shares the list of target browsers between various frontend build tools.

03

EPISODE 03

Laying the Foundation

Q

What is **JSX**?

A

JSX stands for JavaScript XML. JSX allows us to write HTML elements in JavaScript and place them in the DOM without any createElement() and/or appendChild() methods. JSX makes it easier to write and add HTML in React. JSX converts HTML tags into react elements.

Example 1 using JSX:

```
const myElement = <h1>I Love JSX!</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

JSX

Example 2 Without JSX:

```
const myElement = React.createElement('h1', {}, 'I do not use JSX!');
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

JSX

Q

Superpowers of **JSX**.

A

Using JSX, you can write markup inside Javascript, providing you with a superpower to write logic and markup of a component inside a single .jsx file. JSX is easy to maintain and debug.

Example

```
function greeting(user) {
  //JSX
  return <h1>{user}, How are you!!!</h1>;
}
```

JSX

Q Role of `type` attribute in script tag? What options can I use there?

A The `type` attribute specifies the type of the script. The type attribute identifies the content between the `<script>` and `</script>` tags. It has a Default value which is "text/javascript".

`type` attribute can be of the following types:

- `text/javascript` : It is the basic standard of writing javascript code inside the `<script>` tag.

Syntax

```
<script type="text/javascript"></script>
```

HTML

- `text/ecmascript` : this value indicates that the script is following the `EcmaScript` standards.
- `module` : This value tells the browser that the script is a module that can import or export other files or modules inside it.
- `text/babel` : This value indicates that the script is a babel type and required bable to transpile it.
- `text/typescript` : As the name suggest the script is written in `TypeScript` .

Q `{TitleComponent}` vs `{<TitleComponent/>}` vs `{<TitleComponent></TitleComponent>}` in `JSX`.

A The Difference is stated below:

- `{TitleComponent}` : This value describes the `TitleComponent` as a javascript expression or a variable. The `{}` can embed a javascript expression or a variable inside it.
- `<TitleComponent/>` : This value represents a Component that is basically returning Some JSX value. In simple terms `TitleComponent` a function that is returning a JSX value. A component is written inside the `{< />}` expression.
- `<TitleComponent></TitleComponent>` : `<TitleComponent />` and `<TitleComponent></TitleComponent>` are equivalent only when `< TitleComponent />` has no child components. The opening and closing tags are created to include the child components.

Example

```
<TitleComponent>  
  <FirstChildComponent />  
  <SecondChildComponent />  
  <ThirdChildComponent />  
</TitleComponent>
```

JSX

if there are no child elements, then we can write

```
<TitleComponent />
```

JSX

04

EPISODE 04

Talk is Cheap, show me the code

Q Is **JSX** mandatory for React?

A **JSX** is an Extension Syntax that allows writing HTML and Javascript together easily in React and is used to create React elements. These elements are then rendered to the React DOM. Each JSX element is just to make use of React easy and for calling `React.createElement(component, props, ...children)` with less work. So, anything that is done with JSX can also be done with just plain JavaScript. So **JSX** is not mandatory but is used for writing better and clean code instead of writing code using `React.createElement`.

Example of **JSX**

```
const sample = <h2>Greetings</h2>;
```

JSX

Q Is **ES6** mandatory for React?

A **ES6** is not mandatory for **React** but is highly recommendable. The latest projects created on React rely a lot on ES6. React uses ES6, and you should be familiar with some of the new features like: Classes, Arrow Functions, Variables(`let`, `const`). ES6 stands for ECMAScript 6. ECMAScript was created to standardize JavaScript, and ES6 is the 6th version of ECMAScript, it was published in 2015.

Q `{TitleComponent}` vs `<TitleComponent/>` vs `<TitleComponent></TitleComponent>` in **JSX**.

A The Difference is stated below:

- `{TitleComponent}`: This value describes the `TitleComponent` as a javascript expression or a variable or React element. The `{}` can embed a javascript expression or a variable or React element inside it.
- `<TitleComponent/>`: This value represents a Component that is basically returning Some JSX value. In simple terms `TitleComponent` a function that is returning a JSX value. If component is written inside the `{< />}` expression.

- `<TitleComponent></TitleComponent>` : `<TitleComponent />` and `<TitleComponent></TitleComponent>` are equivalent only when `< TitleComponent />` has no child components. The opening and closing tags are created to include the child components.

Example

```
<TitleComponent>
  <FirstChildComponent />
  <SecondChildComponent />
  <ThirdChildComponent />
</TitleComponent>
```

JSX

Q How can I write **comments** in JSX?

A JSX comments are written as follows:

- `{/* */}` - for single or multiline comments

Example

```
{/* A JSX comment */}
{/*
Multi
line
JSX
comment
*/}
```

JSX

Q What is `<React.Fragment></React.Fragment>` and `<></>` ?

A `<React.Fragment></React.Fragment>` is a feature in React that allows you to return multiple elements from a React component by allowing you to group a list of children without adding extra nodes to the DOM. `<></>` is the shorthand tag for `React.Fragment`. The only difference between them is that the shorthand version does not support the key attribute.

Example

```
return (  
  <React.Fragment>  
    <Header />  
    <Navigation />  
    <Main />  
    <Footer />  
  </React.Fragment>  
);  
  
return (  
  <>  
    <Header />  
    <Navigation />  
    <Main />  
    <Footer />  
  </>  
);
```

JSX

Q What is **Reconciliation** in React?

A **Reconciliation** is the process through which React updates the Browser DOM and makes React work faster. React uses a **diffing algorithm** so that component updates are predictable and faster. React would first calculate the difference between the real DOM and the copy of DOM (Virtual DOM) when there's an update of components. React stores a copy of Browser DOM which is called **Virtual DOM**. When we make changes or add data, React creates a new Virtual DOM and compares it with the previous one. Comparison is done by **Diffing Algorithm**. React compares the Virtual DOM with Real DOM. It finds out the changed nodes and updates only the changed nodes in Real DOM leaving the rest nodes as it is. This process is called Reconciliation. (or) Reconciliation is the process by which React updates the UI to reflect changes in the component state. The reconciliation algorithm is the set of rules that React uses to determine how to update the UI in the most efficient way possible. React uses a virtual DOM (Document Object Model) to update the UI.

Q What is **React Fiber**?

A React Fiber is a concept of ReactJS that is used to render a system faster, smoother and smarter. The Fiber reconciler, which became the default reconciler for React 16 and above, is a complete rewrite of React's reconciliation algorithm to solve some long-standing issues in React. Because Fiber is asynchronous, React can:

- Pause, resume, and restart rendering work on components as new updates come in
- Reuse previously completed work and even abort it if not needed
- Split work into chunks and prioritize tasks based on importance

Q Why do we need **keys** in React?

A **key** is a special attribute you need to include when creating lists of elements in React. Keys are used in React to identify which items in the list are changed, updated, or deleted. In other words, we can say that keys are unique Identifier used to give an identity to the elements in the lists. Keys should be given to the elements within the array to give the elements a stable identity.

Example

```
<li key={0}>1</li>
<li key={1}>2</li>
<li key={2}>3</li>
```

J S X

Q Can we use **index as keys** in React?

A Yes, we can use the **index as keys**, but it is not considered as a good practice to use them because if the order of items may change. This can negatively impact performance and may cause issues with component state. Keys are taken from each object which is being rendered. There might be a possibility that if we modify the incoming data react may render them in unusual order.

Q What is **props in React**? Ways to.

A props stands for properties. Props are arguments passed into React components. props are used in React to pass data from one component to another (from a parent component to a child component(s)). They are useful when you want the flow of data in your app to be dynamic.

Example

```
function App() {  
  return (  
    <div className="App">  
      <Tool name="Chetan Nada" tool="Figma"/> // name and tool are props  
    </div>  
  )  
}
```

Q What is **Config Driven UI** ?

A **Config Driven UI** are based on the configurations of the data application receives. It is rather a good practice to use config driven UIs to make application for dynamic. It is a very common & basic approach to interact with the User. It provides a generic interface to develop things which help your project scale well. It saves a lot of development time and effort. A typical login form, common in most of the Apps. Most of these forms also get frequent updates as the requirements increase in terms of Form Validations, dropdown options,.. or design changes.

Q Difference between **Virtual DOM** and **Real DOM** ?

A DOM stands for **Document Object Model**, which represents your application UI and whenever the changes are made in the application, this DOM gets updated and the user is able to visualize the changes. DOM is an interface that allows scripts to update the content, style, and structure of the document.

- **Virtual DOM**

- The Virtual DOM is a light-weight abstraction of the DOM. You can think of it as a copy of the DOM, that can be updated without affecting the actual DOM. It has all the same properties as the real DOM object, but doesn't have the ability to write to the screen like the real DOM.
- Virtual DOM is just like a blueprint of a machine, can do the changes in the blueprint but those changes will not directly apply to the machine.
- Reconciliation is a process to compare and keep in sync the two files (Real and Virtual DOM). Diffing algorithm is a technique of reconciliation which is used by React.

- **Real DOM**

- The DOM represents the web page often called a document with a logical tree and each branch of the tree ends in a node and each node contains object programmers can modify the content of the document using a scripting language like javascript and the changes and updates to the dom are fast because of its tree-like structure but after changes, the updated element and its children have to be re-rendered to update the application UI so the re-rendering of the UI which make the dom slow all the UI components you need to be rendered for every dom update so real dom would render the entire list and not only those item that receives the update .

Real DOM	Virtual DOM
DOM manipulation is very expensive	DOM manipulation is very easy
There is too much memory wastage	No memory wastage
It updates Slow	It updates fast
It can directly update HTML	It can't update HTML directly
Creates a new DOM if the element updates.	Update the JSX if the element update
It allows us to directly target any specific node (HTML element)	It can produce about 200,000 Virtual DOM Nodes / Second.
It represents the UI of your application	It is only a virtual representation of the DOM

05

EPISODE 05

Let's get Hooked!

Q What is the difference between **Named export**, **Default export**, and *** as export**?

A ES6 provides us to import & export a module and use it in other files. ES6 provides two ways to export a module from a file: **named export** and **default export**. In **Named export**, one can have multiple named exports per file. Then import the specific exports they want surrounded in `{ }` braces. The name of imported module has to be the same as the name of the exported module. In **Named export**, the component is exported from MyComponent.js file like:

```
export const MyComponent = () => {}  
export const MyComponent2 = () => {}
```

JSX

and the component is imported from MyComponent.js file like: here we must use `{ }` in MyComponent.

```
// ex. importing a single named export  
import { MyComponent } from "./MyComponent";  
  
// ex. importing multiple named exports  
import { MyComponent, MyComponent2 } from "./MyComponent";  
  
// ex. giving a named import a different name by using "as":  
import { MyComponent2 as MyNewComponent } from "./MyComponent";
```

JSX

In **Default export**, One can have only one default export per file. The naming of import is completely independent in default export and we can use any name we like. In **Default export**, the component is exported from MyComponent.js file like:

```
const MyComponent = () => {}  
export default MyComponent;
```

JSX

and the component is imported from MyComponent.js file like: here we must omit `{ }` in MyComponent.

```
import MyComponent from "./MyComponent";
```

JSX

In `* as export`, it is used to import the whole module as a component and access the components inside the module. In `* as export`, the component is exported from MyComponent.js file like:

```
export const MyComponent = () => {}  
export const MyComponent2 = () => {}  
export const MyComponent3 = () => {}
```

JSX

and the component is imported from MyComponent.js file like:

```
import * as MainComponents from "./MyComponent";
```

JSX

Now we can use them in JSX as:

```
<MainComponents.MyComponent />  
<MainComponents.MyComponent2 />  
<MainComponents.MyComponent3 />
```

JSX

We can use **Named export** and **Default export** together. So you should export like:

```
export const MyComponent2 = () => {}  
const MyComponent = () => {}  
export default MyComponent;
```

JSX

and import like:

```
import MyComponent, {MyComponent2} from "./MyComponent";
```

JSX

Q What are React Hooks?

A In React version 16.8, React introduced a new pattern called Hooks. React Hooks are simple JavaScript functions that we can use to isolate the reusable part from a functional component. Hooks can be stateful and can manage side-effects. Hooks allow you to reuse stateful logic without changing your component hierarchy. This makes it easy to share Hooks among many components or with the community.

React provides a bunch of standard in-built hooks:

- `useState`: To manage states. Returns a stateful value and an updater function to update it.
- `useEffect`: To manage side-effects like API calls, subscriptions, timers, mutations, and more.
- `useContext`: To return the current value for a context.
- `useReducer`: A `useState` alternative to help with complex state management.
- `useCallback`: It returns a memorized version of a callback to help a child component not re-render unnecessarily.
- `useMemo`: It returns a memoized value that helps in performance optimizations.
- `useRef`: It returns a ref object with a current property. The ref object is mutable. It is mainly used to access a child component imperatively.
- `useLayoutEffect`: It fires at the end of all DOM mutations. It's best to use `useEffect` as much as possible over this one as the `useLayoutEffect` fires synchronously.
- `useDebugValue`: Helps to display a label in React DevTools for custom hooks.

Q Why do we need `useState Hook` ?

A `useState hook` is used to maintain the state in our React application. It keeps track of the state changes so basically `useState` has the ability to encapsulate local state in a functional component. The `useState` hook is a special function that takes the `initial state` as an `argument` and `returns an array` of two entries. `useState` encapsulate only singular value from the state, for multiple state need to have `useState` calls.

Syntax for `useState` hook

```
const [state, setState] = useState(initialstate);
```

JSX

Importing: To use `useState` you need to import `useState` from `react` as shown below:

```
import React, { useState } from "react";
```

JSX

we can use Hooks in Functional Components

```
const Example = (props) => {  
  // You can use Hooks here!  
  return <div />;  
}
```

06

EPISODE 06

Exploring the world

Q What is **Microservice** ?

A **Microservice** - also known as the microservice architecture - is an architectural and organizational approach to software development where software is composed of small independent services like database, server or a UI of the application, that communicate over well-defined APIs. These services are owned by small, self-contained teams. Microservices architectures make applications easier to scale and faster to develop, enabling innovation and accelerating time-to-market for new features. means we are dividing software into small, well-defined modules enables teams to use functions for multiple purposes.

- Benefits of Microservices:
 - Flexible Scaling
 - Easy Deployment
 - Technological Freedom
 - Reusable Code
 - Resilience

Q What is **Monolith architecture** ?

A **Monolith architecture** is a traditional model of a software program, which is built as a unified unit that is self-contained and independent from other applications. A monolithic architecture is a singular, large computing network with one code base that couples all of the business concerns together. To make a change to this sort of application requires updating the entire stack by accessing the code base and building and deploying an updated version of the service-side interface. This makes updates restrictive and time-consuming. means we are not dividing software into small, well-defined modules, we use every services like, database, server or a UI of the application, in one Application file.

Q What is the difference between Monolith and Microservice ?

A With **monolithic architectures**, all processes are tightly coupled and run as a single service. This means that if one process of the application experiences a spike in demand, the entire architecture must be scaled. Adding or improving a monolithic application's features becomes more complex as the code base grows. This complexity limits experimentation and makes it difficult to implement new ideas. Monolithic architectures add risk for application availability because many dependent and tightly coupled processes increase the impact of a single process failure.

With a **microservices architecture**, an application is built as independent components that run each application process as a service. These services communicate via a well-defined interface using lightweight APIs. Services are built for business capabilities and each service performs a single function. Because they are independently run, each service can be updated, deployed, and scaled to meet demand for specific functions of an application.

Q Why do we need a useEffect Hook ?

A **useEffect Hook** is javascript function provided by **react**. The useEffect Hook allows you to **eliminate side effects** in your components. Some examples of side effects are: **fetching API data**, **directly updating the DOM**, and **setting up subscriptions or timers**, etc can be lead to unwarranted side-effects. useEffect accepts **two arguments**, a **callback function** and a **dependency array**. The second argument is optional.

```
useEffect(() => {}, [])
```

J S X

The **() => {}** is callback function and **[]** is called a empty dependency array. If anything that we pass (suppose currentState) inside the **[]** it trigger the callback function and changes the state of the application.

```
useEffect(() => {  
  setCurrentState("true");  
}, [currentState])
```

J S X

If we do not pass empty dependency array then the useEffect runs everytime when the UI is rendered.

```
useEffect(() => {})
```

J S X

Q What is **Optional Chaining** ?

A **Optional Chaining** (`?.`) operator accesses an object's property or calls a function. If the object accessed or function called is **undefined or null**, it returns **undefined** instead of throwing an error. **Optional Chaining** (`?.`) is good way of accessing the object keys, it prevents the application from being crashed if the key that we are trying to access is not present. If the key is not present then instead of a throwing key error, it returns **undefined**.

Q What is **Shimmer UI** ?

A A **Shimmer UI** resembles the page's actual UI, so users will understand how quickly the web or mobile app will load even before the content has shown up. It gives people an idea of what's about to come and what's happening (while UI currently loading) when a page full of content/data takes more than 3 - 5 seconds to load. Shimmer UI is a great way for loading the applications. Instead of showing a loading circle we can design a shimmer UI for our application that is good for user experience.

Q What is the difference between **JS expression** and **JS statement** ?

A A **JS expression** returns a value that we use in the application. for example:

```
1 + 2 // expresses
"foo".toUpperCase() // expresses 'F00'
console.log(2) // logs '2'
isTrue ? true : false // returns us a true or false value based on isTrue value
```

JSX

A **JS statement**, does not return a value. for example:

```
let x; // variable declaration
if () { } // if condition
```

JSX

If we want to use **JS expression** in JSX, we have to wrap in `{/* expression slot */}` and if we want to use **JS statement** in JSX, we have to wrap in `{(/* statement slot */) }`;

Q What is **Conditional Rendering** ? explain with a code example.

A **Conditional rendering** in React works the same way conditions work in **JavaScript**. Use JavaScript operators like **if** or the **conditional operator** to create elements representing the current state, and let React update the UI to match them. for example:

```

// Using Ternary operator as a shorthand way of writing an if-else statement
{isLoggedIn ? (return <UserGreeting />) : (return <GuestGreeting />)};
// Using an if...else Statement
{
  (if (isLoggedIn) {
    return <UserGreeting />;
  }else {
    return <GuestGreeting />;
  })
}
// Using Logical &&
{isLoggedIn && <button>Logout</button>}

```

JSX

Q What is CORS ?

A Cross-Origin Resource Sharing (CORS) is an HTTP-header based mechanism that allows a server to indicate any origins (domain, scheme, or port) other than its own from which a browser should permit loading resources. CORS defines a way in which a browser and server can interact to determine whether it is safe to allow the cross-origin request.

Q What is async and await ?

A **Async** : It simply allows us to write promises-based code as if it was synchronous and it checks that we are not breaking the execution thread. It operates asynchronously via the event loop. Async functions will always return a promise. It makes sure that a promise is returned and if it is not returned then JavaScript automatically wraps it in a promise which is resolved with its value. **Await** : Await function is used to wait for the promise. It could be used within the **async** block only. It makes the code wait until the promise returns a result. It only makes the async block wait. for example:

```

// async function getRestaurant to fetch Swiggy API data
async function getRestaurants() {
  const data = await fetch(
    "Swiggy_API_URL"
  );
  const json = await data.json();
  // we get the Swiggy API data in json format
  console.log(json);
}

```

JSX

Q What is the use of `const json = await data.json();` in `getRestaurants()`?

A The `data` object, returned by the `await fetch()`, is a generic placeholder for multiple data formats. so we can extract the `JSON object` from a `fetch` response by using `await data.json()`. `data.json()` is a method on the data object that lets you extract a `JSON object` from the data or response. The method returns a promise because we have used `await` keyword. so `data.json()` returns a promise resolved to a `JSON object`.

07

EPISODE 07

Finding the Path

Q What are various ways to **add images** into our App? Explain with **code examples**.

A Using the **full URL of the image** for the web (CDN) or any public images. Example :

```

```

JSX

Adding the image into the project **Drag your image into your project** and **import it** into the desired component

```
import reactLogo from "./reactLogo.png";
export default function App() {
  return <img src={reactLogo} alt="react logo" />
}
```

JSX

The correct way to structure images in your project is to add them in an **images** folder. If you are using other **assets** than just images, you might want to add all in the **assets** folders.

```
import reactLogo from "../../assets/images/reactLogo.png";
export default function App() {
  return <img src={reactLogo} alt="react logo" />
}
```

JSX

Q What would happen if we do **console.log(useState())** ?

A If we do **console.log(useState())**, we get an array **[undefined, function]** where first item in an array is **state** is **undefined** and the second item in an array is **setState** **function** is bound dispatchSetState.

Q How will **useEffect** behave if we **don't add** a **dependency array** ?

A Syntax of **useEffect** is:

```
useEffect(() => {}, []);
```

J S X

Case 1 : When the **dependency array is not included** in the arguments of **useEffect() hook**, the callback function will be executed **every time** the component is rendered and re-rendered.

```
useEffect(() => {  
    console.log("I run everytime this component rerenders")  
});
```

J S X

Case 2 : When the **dependency array is empty** in the arguments of **useEffect() hook**, the callback function will be executed **only one time** during the initial render of the component.

```
useEffect(() => {  
    console.log("I Only run once (When the component gets mounted)")  
}, []);
```

J S X

Case 3 : When the **dependency array contains a condition**, the callback function will be executed **one time** during the initial render of the component and also rerender if there is a **change in the condition**.

```
useEffect(() => {  
    console.log("I run every-time when my condition changed")  
}, [condition]);
```

J S X

Q What is SPA ?

A **Single Page Application (SPA)** is a web application that dynamically updates the webpage with data from web server without reloading/refreshing the entire page. All the HTML, CSS, JS are retrieved in the initial load and other data/resources can be loaded dynamically whenever required. An SPA is sometimes referred to as a **single-page interface (SPI)**.

Q What is the difference between **Client Side Routing** and **Server Side Routing** ?

A In **Server-side routing or rendering (SSR)**, every change in URL, http request is made to server to fetch the webpage, and replace the current webpage with the older one.

In **Client-side routing or rendering (CSR)**, during the first load, the webapp is loaded from server to client, after which whenever there is a change in URL, the router library navigates the user to the new page without sending any request to backend. All **Single Page Applications uses client-side routing**.

08

EPISODE 08

Let's get Classy

Q What is the order of life cycle method calls in **Class Based Components** ?

A Following is the order of lifecycle methods calls in **Class Based Components** :

1. constructor()
2. render ()
3. componentDidMount()
4. componentDidUpdate()
5. componentWillUnmount()

For more reference [React-Lifecycle-methods-Diagram](#)

Q Why do we use **componentDidMount** ?

A The **componentDidMount()** method allows us to execute the React code when the component is already placed in the DOM (Document Object Model). This method is called during the Mounting phase of the React Life-cycle i.e after the component is rendered. Wwe can run any piece of react code to modify the components. For ex. It's the best place to **make API calls** .

Q Why do we use **componentWillUnmount** ? Show with example.

A **componentWillUnmount()** is useful for the cleanup of the application when we switch routes from one place to another. Since we are working with a SPA(Single Page Application) the component process always runs in the background even if we switch to another route. So it is required to stop those processes before leaving the page. If we revisit the same page, a new process starts that affects the browser performance. For example, in Repo class, during **componentDidMount()** a timer is set with an interval of every one second to print in console. When the component is unmounted (users moves to a different page), the timer will be running in the background, which we might not even realize and causing huge performance

issue. To avoid such situations the cleanup function can be done in `componentWillUnmount`, in this example `clearInterval` (timer) to clear the timer interval before unmounting Repo component.

Q (Research) Why do we use `super(props)` in constructor?

A `super(props)` is used to inherit the properties and access of variables of the React parent class when we initialize our component. `super()` is used inside constructor of a class to derive the parent's all properties inside the class that extended it. If `super()` is not used, then Reference Error : Must call super constructor in derived classes before accessing 'this' or returning from derived constructor is thrown in the console. The main difference between `super()` and `super(props)` is the `this.props` is undefined in child's constructor in `super()` but `this.props` contains the passed props if `super(props)` is used.

Q (Research) Why can't we have the callback function of `useEffect` `async`?

A `useEffect` expects its callback function to return nothing or return a function (cleanup function that is called when the component is unmounted). If we make the callback function as `async`, it will return a `promise` and the promise will affect the clean-up function from being called.

13

EPISODE 13

Time for the Test

Theory Assignment:

- What are the difference types of Testing?
- React Testing Library and It's set up
- What is Jest and why do we use it?
- Jest setup and installation of it's related

Types of testing (developer)

- Unit Testing - Testing One Component in Isolation | | means seperately
- Integration Testing - Testing Integration of Components
- End to End Testing (or) e2e Testing - End-to-end testing verifies that all components of a system can run under real-world scenarios. The goal of this form of testing is to simulate a user experience from start to finish. E2E testing can find software dependencies while also validating the system under test, its data integrity and integrations.

Setting up Testing in our app

- Install React Testing Library
- Install Jest
- Install Babel Dependencies
- Configure Babel
- Configure Parcel Config File to disable default Babel transpilation
- Jest - `npx jest --init`
- Install jsdom library
- Install @babel/preset-react - to make JSX work in test cases
- Include @babel/preset-react inside my babel config
- `npm i -D @testing-library/jest-dom`